

Reverse Engineering Client-Side Movement in World of Warcraft 3.3.5a: A Definitive Analysis of Click-To-Move Interruptions and State Desynchronization

Abstract and Executive Summary

The analysis of client-server architecture within massively multiplayer online role-playing games (MMORPGs) reveals that client-side movement prediction and network synchronization represent some of the most complex subsystems in game development. The World of Warcraft client, specifically build 12340 (version 3.3.5a), utilizes a highly deterministic movement engine that blends hardware input polling with automated, path-based navigation mechanisms.¹ For software engineers and security researchers engineering dynamic-link libraries (DLLs) injected into the 32-bit client memory space, manipulating the `CGPlayer_C` object directly offers unprecedented control over the local player's spatial state. However, altering high-level state variables without correctly interfacing with the underlying physics and movement state machine frequently results in severe desynchronization between the local client and the authoritative server.

This exhaustive research report delivers a forensic analysis of the Click-To-Move (CTM) architecture in the 3.3.5a build 12340 client. It systematically addresses the precise methodologies required to cleanly intercept and cancel CTM pathing. By dissecting the memory layouts, state transitions, spline interpolation mathematics, and network serialization protocols, this report demystifies the root causes of runaway movement loops and infinite heartbeat transmission anomalies. Furthermore, it details the precise internal function pointers, such as `StopCTM`, and the memory offsets necessary to execute movement cancellations safely, ensuring absolute parity between the client's visual representation and the server's tracking heuristics.

Introduction to Client-Server Movement Architecture

In standard networked game environments, the architecture dictates a delicate balance between client-side prediction and server-side authority. The World of Warcraft 3.3.5a client engine operates on a hybrid model. To ensure smooth gameplay and immediate responsiveness, the client is granted the authority to calculate its own movement trajectories and immediately update the local rendering loop. Simultaneously, it must consistently serialize this spatial data into network packets—known as opcodes—and transmit them to the server.

The server, often running on emulators like TrinityCore or MaNGOS for the 3.3.5a ecosystem², receives these packets, validates the mathematical feasibility of the movement, and replicates the state to other connected clients.

The Click-To-Move (CTM) subsystem is an automated pathfinding and movement execution engine embedded natively within the client.⁴ Unlike standard hardware-driven movement—where discrete key presses correspond directly to the setting and clearing of specific bitwise flags in the player's movement state—CTM relies on calculating a mathematical spline. Once the destination vector is established, the client autonomously drives the player character along this trajectory until a predefined termination condition is mathematically satisfied.

When an external DLL manipulating the client's memory space calls the internal ClickToMove function at address 0x00727400⁵, it injects a new trajectory into this state machine. The primary engineering challenge arises not in initiating this autonomous movement, but in prematurely aborting it. Halting the character prior to the natural completion of the route requires a nuanced understanding of how the client maintains its internal MovementInfo structure, how it formats movement opcodes such as MSG_MOVE_STOP (0x00CF), and how it communicates these instantaneous kinetic changes to the server's authoritative tracking systems.

The Mechanics of Programmatic Autonomous Navigation

Before analyzing the teardown process, one must comprehensively understand how autonomous movement is constructed within the client's memory space. The traditional method of navigating a 3D environment involves binding physical hardware inputs to virtual game states.⁶ When a user depresses the "W" key, the client's virtual key state manager flags the forward movement intent, updating the character's internal kinetic vector. This relies heavily on the user's manual adjustments of the camera and orientation.⁷

Conversely, the Click-To-Move system abstracts this mechanical input into a programmable interface.⁴ Originally designed to allow users with specific accessibility needs or playstyle preferences to navigate simply by interacting with the terrain⁹, CTM has become the foundational pillar for automation frameworks, combat rotation engines, and automated fishing protocols like Fishbot-3.3.5.¹¹ By calling the internal functions directly via an injected C++ DLL, developers can bypass the user interface entirely, instructing the client to calculate its own navmesh interpolations.¹

The function signature for initiating this movement is explicitly defined within the client's binary structure:

```
void __thiscall CGPlayer_C::ClickToMove(int action, uint64_t* guid, Vec3* pos, float precision)
```

This function utilizes the `__thiscall` calling convention, a standard in Microsoft Visual C++ compiled 32-bit binaries, which mandates that the pointer to the calling object (this, representing the `CGPlayer_C` base address) is passed via the ECX CPU register. The parameters strictly define the parameters of the autonomous task:

- **Action:** A 32-bit integer defining the behavioral intent (e.g., move, attack, interact).
- **GUID:** A pointer to a 64-bit unsigned integer representing the target's Globally Unique Identifier, utilized when the action involves a specific entity rather than a static coordinate.
- **Pos:** A pointer to a 3D vector structure (`Vec3`) containing the X, Y, and Z floating-point coordinates.
- **Precision:** A floating-point value establishing the radius of acceptance—the distance threshold at which the client considers the destination reached and terminates the spline.

The Anatomy of the Click-To-Move Subsystem and Memory Layout

To comprehend why standard interruptions fail, one must dissect the physical memory layout of the CTM subsystem within the `CGPlayer_C` object. The CTM framework is not merely a transient function call; it instantiates a persistent data block that dictates the player's autonomous navigation parameters across multiple rendering frames.⁴

Memory Layout of the CTM Block

Historical reverse engineering of the 3.x client branch reveals that the Click-To-Move mechanism relies on a dedicated contiguous memory block.⁴ This block stores the operational parameters of the current CTM task. The precise offsets relative to the base of the CTM structure for the 3.3.5a build are presented in Table 1 below.

Field Description	Offset (Hex)	Data Type	Functionality / Detail Analysis
Turn Scale	0x04	float	Determines the scaling factor for character rotation when adjusting the facing angle (yaw) to the destination vector. ⁴

Interaction Distance	0x0C	float	Defines the threshold radius from the targeted spatial position at which the client automatically halts the kinetic spline. ⁴
Action Type	0x1C	int32	The active state trigger determining the nature of the movement, dictating whether the engine should open a loot window, attack, or simply stand idle. ⁴
Target GUID	0x20	uint64_t	The 64-bit identifier of the target entity. Used extensively for dynamic pathing to moving targets. ⁴
Destination X	0x74	float	The absolute Cartesian X-coordinate of the spatial destination. ⁴
Destination Y	0x78	float	The absolute Cartesian Y-coordinate of the spatial destination. ⁴
Destination Z	0x7C	float	The absolute Cartesian Z-coordinate (elevation) of the spatial destination. ⁴

When the client executes the ClickToMove function at 0x00727400⁵, it allocates and populates this memory block. Once populated, the client's main engine update loop intercepts these values and calculates a navigation path. It utilizes a spline-based interpolation algorithm to

iteratively update the character's coordinates frame-by-frame.

The Enumeration of Action Types

The action parameter at offset 0x1C is the primary behavioral pivot for the CTM engine.⁴ The WoW client defines a strict enumeration for these actions, dictating how the character behaves upon reaching the destination. Table 2 outlines the established action values and their engine-level interpretations.

Hex Value	Action Name	Behavioral Description
0x01	LeftClick	Navigates to a targeted coordinate without invoking a contextual interaction trigger. ⁴
0x02	Face	Modifies the character's yaw vector to face a specific coordinate or entity without generating translation. ⁴
0x03	Stop	Historically utilized to halt movement, though heavily deprecated. Known to throw Lua execution errors in later builds (e.g., WoD 17658). ⁴
0x04	Move	The standard programmatic command to generate a physical translation spline to a coordinate. ⁴
0x05	NpcInteract	Generates a spline to an NPC and automatically triggers the gossip or vendor interface upon proximity. ⁴
0x06	Loot	Navigates to a slain entity

		and hooks into the internal LootWindowOffset (0x00BFA8D8 in 12340). ⁴
0x07	ObjectInteract	Directs the character to interact with a static world object, such as a gathering node or quest object. ⁴
0x09	Skin	Navigates to a skinnable entity and triggers the tradeskill interaction. ⁴
0x0A	AttackPosition	Moves the character to a specific location while maintaining an aggressive combat posture. ⁴
0x0D	None / Idle	Represents the null state, indicating the CTM engine has no active task. ⁴

Crucially, as soon as any kinetic action (such as 0x04 Move) is invoked and the spline is generated, the client automatically applies a bitwise modification to the local player's MovementInfo structure, asserting the FORWARD movement flag (0x00000001).¹² This flag operates independently of the CTM memory block and serves as the definitive signal to the network replication layer that the character is physically in motion.

Diagnosing Movement Interruption Failures

When a software engineer attempts to programmatically abort an ongoing CTM route—perhaps because a combat target has perished or a higher-priority task has emerged in a custom rotation engine¹—the intuitive approach is to overwrite the CTM state block or simulate a keyboard stop command. However, empirical testing demonstrates that these methodologies consistently result in catastrophic client-server desynchronization. The client continues to mathematically interpolate the player's position, physically moving the character forward on the screen, while simultaneously spamming MSG_MOVE_HEARTBEAT (0x00EE) packets to the server.

Analyzing the explicit failure modes of these common heuristics provides vital insight into the client's internal architecture and answers why partial state mutations fail.

Failure Mode 1: Modifying the Action State to Idle (0x0D)

The most frequent attempt to halt CTM is issuing a new command to overwrite the state: `CallCTM(action=0x0D,...)`.⁴ While passing Action 0x0D correctly updates the Action Type at memory offset 0x1C to the "None" state, it fundamentally fails to arrest the kinetic physics.

The internal `ClickToMove` function is an initializer, not an interceptor. When it receives an Idle command, it updates the high-level UI intent but entirely bypasses the logic required to signal the underlying physics and spline interpolator to halt execution. Because the physical spline remains allocated in memory, the game's core rendering loop continues to calculate forward translation along that curve. Consequently, the FORWARD flag within the deeply embedded `MovementInfo` structure remains asserted.¹² The network layer, reading this asserted flag, correctly deduces that the character is still translating through space and continues to broadcast `MSG_MOVE_HEARTBEAT` packets. Because the physical movement logic was never explicitly told to tear down the spline, a `MSG_MOVE_STOP` (0x00CF) packet is never constructed or transmitted.

Failure Mode 2: The Deprecated Stop Command (0x03)

Similarly, invoking `CallCTM(action=0x03,...)` yields identical runaway behavior. While 0x03 historically signifies "Stop" in earlier iterations of the engine, it lacks the necessary hooking in the 3.3.5a engine to force a spline teardown. In later client architectures, such as Warlords of Draenor build 17658, passing this action outright causes the engine to throw unhandled Lua errors.⁴ In 12340, it simply fails silently, leaving the mathematical spline intact and the character running indefinitely.

Failure Mode 3: Overwriting Destination Coordinates with High Precision

Another highly analytical but flawed approach involves issuing a new Move command (0x04) with the destination vector set precisely to the character's exact current spatial coordinates, utilizing an extremely high precision tolerance (e.g., `precision=100000`). The theoretical hypothesis is that the client's pathfinding mesh will instantly calculate the distance to the destination as zero, satisfy the termination condition, and cleanly destroy the spline.

In practice, the WoW 3.3.5a client's floating-point mathematics and navmesh processing do not interpret this optimally. When the engine recalculates a path to the current position while a character is already possessing momentum, it attempts to plot a trajectory to a point that is instantly behind the character's kinetic bounding box. Normalizing this near-zero delta vector frequently results in floating-point anomalies or micro-stutters, forcing the character into an infinite run-in-place state. The termination condition is paradoxically never met, the FORWARD flag remains active¹², and no `MSG_MOVE_STOP` is ever dispatched to the server.

Failure Mode 4: Lua API Execution and Virtual Key States

Executing arbitrary Lua strings such as `MoveForwardStop()` via internal functions like

FrameScript_Execute (at 0x00819210) appears entirely logical, as it is the native method the user interface utilizes for halting keyboard-driven movement. However, the engine's architecture cleanly separates hardware input states from autonomous CTM states.

The Lua API binds directly to the virtual key state manager. When MoveForwardStop() is invoked by the scripting engine, it clears the virtual key flag for moving forward. If the character was translating exclusively because the user was holding down the "W" key, this successfully forces the FORWARD flag in the MovementInfo struct to clear and triggers a legitimate MSG_MOVE_STOP. However, if the movement was initiated by the CTM engine, the autonomous state machine actively forces the FORWARD flag to remain true on every single frame update, actively overriding the virtual key state manager.¹ Thus, the Lua command's effect is instantly negated by the CTM engine's subsequent tick. Even combining these calls in the same frame fails, as the underlying spline interpolator maintains absolute authority over the character's physics until it is explicitly destroyed.

The Pathology of the MSG_MOVE_HEARTBEAT Loop

The aforementioned failures all culminate in a specific network anomaly: the infinite transmission of MSG_MOVE_HEARTBEAT (0x00EE) packets. The World of Warcraft protocol relies on an asynchronous movement replication model. When physical movement is initiated, the client sends a start opcode. As long as the FORWARD flag (0x00000001)¹² remains active, the client relies on the server to natively interpolate the player's position based on their velocity.

However, to prevent massive desynchronization due to network latency, the client implements a heartbeat mechanism. If the state machine detects active movement, it routinely dispatches a MSG_MOVE_HEARTBEAT containing the player's current coordinates. When a DLL ineffectively attempts to stop the character by overwriting the CTM action to 0x0D⁴, the UI state is "Idle," but the physical movement spline remains untouched. The client sees the FORWARD flag is active, generating heartbeats indefinitely. Because no MSG_MOVE_STOP (0x00CF) is sent, the server views the player as continuously running, leading to severe positional desynchronization and potential flagging by server-side anti-cheat algorithms.

The Internal Resolution: Function StopCTM

The architectural discrepancies highlighted above demonstrate unequivocally that halting an autonomous movement in build 12340 requires a function explicitly designed to interface with all levels of the engine simultaneously. The client must tear down the mathematical spline, reset the CTM memory block, interface with the MovementInfo struct to clear the physical movement flags, and construct the appropriate network payload. In the 3.3.5a client, this functionality is entirely encapsulated within a dedicated internal function: StopCTM.

Locating and Executing the Teardown Sequence

Extensive reverse engineering of the 12340 binary, corroborated by automation framework

architectures such as IceFlake⁵, reveals that the StopCTM function is located at the memory address 0x0072B3A0. This function operates directly on the local player object and is specifically responsible for executing a clean, multi-layered teardown of the Click-To-Move state machine.

To utilize this function securely from an injected C++ DLL, software engineers must cast this static address to a function pointer utilizing the `__thiscall` calling convention. This ensures that the ECX CPU register correctly contains the CGPlayer_C local player base address prior to execution.

```
typedef void(__thiscall* StopCTM_fn)(void* localPlayer);
```

```
StopCTM_fn StopCTM = (StopCTM_fn)0x0072B3A0;
```

The execution of StopCTM initiates a highly specific, deterministic sequence of internal operations:

1. **Spline Destruction:** The function immediately deallocates the active mathematical spline driving the character's spatial interpolation. The vector physics are instantly nullified.
2. **Memory Block Reset:** It accesses the CTM memory block directly and forcibly overwrites the Action Type at offset 0x1C⁴ to 0x0D (Idle). It simultaneously zeroes out the target coordinates and the Interacting GUID at offset 0x20.⁴
3. **Flag Clearing:** It interfaces deep within the CGUnit_C underlying MovementInfo structure, applying bitwise NOT AND (~) operations to clear the kinetic flags, most notably the FORWARD (0x00000001)¹² and rotational turning flags.
4. **Network Dispatch:** Most critically, StopCTM invokes the internal packet construction protocols to format a MSG_MOVE_STOP (0x00CF) payload. It reads the character's exact frozen spatial coordinates, packages them into a CDataStore buffer alongside the timestamp and orientation, and pushes the packet into the client's NetClient::Send queue.

This deterministic teardown ensures that both the visual client state and the server's authoritative state are instantaneously synchronized, eliminating the infinite MSG_MOVE_HEARTBEAT anomaly entirely.

Validating Execution Context with IsClickMoving

In highly optimized rotation bots or real-time navigation meshes¹, executing the StopCTM function blindly on every rendering frame can incur unnecessary CPU overhead. Furthermore, excessive internal state mutations can potentially flag server-side Warden heuristics due to anomalous memory access patterns. Best practices in reverse engineering dictate verifying the character's actual CTM status before attempting an interrupt.

The WoW client provides an internal boolean evaluation function specifically for this purpose, IsClickMoving, located at 0x00721F90.⁵ By defining a `__thiscall` function pointer to this address,

developers can implement a highly efficient conditional heuristic:

C++

```
typedef bool(__thiscall* IsClickMoving_fn)(void* localPlayer);
IsClickMoving_fn IsClickMoving = (IsClickMoving_fn)0x00721F90;

if (IsClickMoving(player_base)) {
    StopCTM(player_base);
}
```

This conditional check ensures that the intensive teardown logic—and subsequent network dispatches—are only executed when a mathematically active spline is currently driving the character, preserving engine stability.

Structural Analysis of MovementInfo and MovementFlags

To achieve a comprehensive mastery of the WoW 3.3.5a movement architecture, one must intimately understand the lower-level data structures that StopCTM manipulates. Even if a developer successfully utilizes the StopCTM function pointer 0x0072B3A0⁵, understanding the MovementInfo architecture is critical for debugging advanced desynchronization issues and engineering custom packet manipulation.

Within the CGUnit_C base class (from which CGPlayer_C inherits), the MovementInfo structure acts as the definitive source of truth for the entity's spatial, kinetic, and temporal state. This dense memory block contains absolute Cartesian coordinates, yaw/pitch orientation, network timestamp data, and a heavily utilized bitmask defining the active kinetic state, known natively as MovementFlags.

The MovementFlags Bitmask

The MovementFlags field is a 32-bit unsigned integer where each individual bit represents a specific kinetic or environmental state. Table 3 details the critical bit values for the 3.3.5a build, extrapolated from historical data arrays and emulation frameworks.¹²

Movement Flag	Hexadecimal	Bit Position	Technical
---------------	-------------	--------------	-----------

Constant	Value		Description
FORWARD	0x00000001	Bit 0	Character is translating forward along its primary vector. ¹²
BACKWARD	0x00000002	Bit 1	Character is translating backward. ¹²
STRAFE_LEFT	0x00000004	Bit 2	Character is translating laterally to the left. ¹²
STRAFE_RIGHT	0x00000008	Bit 3	Character is translating laterally to the right. ¹²
TURN_LEFT	0x00000010	Bit 4	Character is rotating left (altering yaw). ¹²
TURN_RIGHT	0x00000020	Bit 5	Character is rotating right (altering yaw). ¹²
IS_FALLING	0x00002000	Bit 13	Character has lost ground contact and is accelerating via engine gravity. ¹²
IS_SWIMMING	0x00200000	Bit 21	Character is translating through a mapped fluid volume. ¹²
ON_TRANSPORT	0x02000000	Bit 25	Character coordinates are relative to a dynamic transport entity (e.g., ships,

			zeppelins). ¹²
SPLINE_ELEVATION	0x04000000	Bit 26	Character is moving along a strict Z-axis elevation spline (e.g., automated flight paths). ¹²

When ClickToMove(0x04,...) is invoked ⁴, the client's internal physics engine automatically applies a bitwise OR operation to set the FORWARD flag: MovementFlags |= 0x00000001;

Server-Side Validation and Client-Side Macros

The client's internal validation systems, frequently utilizing the HasMovementFlag macro ¹⁴, constantly poll this 32-bit integer. For instance, the internal logic evaluates HasMovementFlag(MovementFlags(MOVEFLAG_FALLING | MOVEFLAG_FALLINGFAR)) ¹⁴ to calculate potential fall damage thresholds upon impact, or to disable specific spellcasting abilities while airborne. Similarly, the SpellInterrupts_InterruptFlags enumeration defines how certain auras are immediately cancelled upon the detection of movement.¹³

If an external DLL attempts to manipulate the spatial coordinates X, Y, Z at CTM offsets 0x74, 0x78, 0x7C ⁴ without simultaneously addressing the MovementFlags bitmask, the client's predictive engine detects a fatal discrepancy. This discrepancy triggers rubberbanding, wherein the server forcefully rejects the anomalous local coordinates and teleports the client back to its last known valid position.

Direct Memory Manipulation of Movement Flags

A critical question arises: can a developer manually clear these flags without utilizing the StopCTM function? If a developer operates in an environment where calling StopCTM at 0x0072B3A0 ⁵ is strictly unfeasible—perhaps due to hooking constraints, execution context limitations, or anti-cheat monitoring on that specific execution address—halting the character requires manually replicating the entire teardown sequence in C++.

To achieve this, the engineer must locate the exact MovementFlags offset within the MovementInfo struct embedded in the unit base. While the exact offset fluctuates between minor client revisions, it is consistently located deep within the unit's contiguous memory block. To manually halt the player, the following operations must be flawlessly executed:

1. **Clear the Kinetic Bitmask:** Read the 32-bit MovementFlags integer from memory. Apply a bitwise AND with the inverse of the directional flags, and write the new value back to the memory address. MovementFlags &= ~(0x00000001 | 0x00000002 | 0x00000004 | 0x00000008);.¹²
2. **Zero the CTM Action:** Access the CTM block and write 0x00000000 or 0x0D to the

Action Type offset at 0x1C.⁴ Zero the destination coordinates 0x74, 0x78, 0x7C.⁴

3. **Construct the Network Payload:** A manual memory manipulation does not natively notify the server of the kinetic change. The developer must manually instantiate a CDataStore object, populate it with the MSG_MOVE_STOP opcode (0x00CF), append the newly modified MovementInfo structure, calculate the precise millisecond timestamp, calculate fall timers if IS_FALLING was active, and pass the buffer to the client's NetClient::Send function.

Because of the extreme mathematical complexity of correctly calculating these timestamps and fall timers within the MovementInfo network payload, manual manipulation is highly discouraged and deeply prone to error. The StopCTM function fundamentally encapsulates all of these operations safely, natively, and accurately.

Network Protocol, Opcodes, and Movement Synchronization

The consequences of failing to properly utilize StopCTM extend far beyond local client visual glitches; they manifest primarily at the network synchronization layer. The World of Warcraft server infrastructure relies completely on opcodes to maintain world state.

The Critical Role of MSG_MOVE_STOP (0x00CF)

In the emulation frameworks widely utilized for 3.3.5a servers, such as TrinityCore and MaNGOS, movement opcodes are strictly enforced and routed through heavy validation.² While opcodes can vary significantly by specific sub-patch (e.g., MaNGOS mapping stop to 0x0B7 in certain iterations², while standard 3.3.5a packet structures often map it to 0x00CF or similar designations depending on the exact build context³), the protocol mandate remains identical: the server must receive a formal termination command to arrest spatial translation on the server-side navmesh.

When the StopCTM function (0x0072B3A0)⁵ is natively invoked, it inherently handles the construction of this stop packet. The packet structure generated requires immense precision:

- A 16-bit payload size header.
- The specific Opcode (0x00CF).
- The Packed GUID of the player.
- The updated 32-bit MovementFlags mask.¹²
- The GetTickCount() based millisecond timestamp.
- The exact X, Y, Z floating-point coordinates.
- The player's orientation (Yaw).

By natively formatting this payload, StopCTM ensures that the server's tracking aligns precisely with the client's rendered position.

Emulator Tracking and Desynchronization Penalties

Server emulators do not passively accept coordinates. They calculate the Euclidean distance between the last known coordinate and the new coordinate, divide it by the elapsed time between timestamps, and compare the result against the character's known movement speed (e.g., run speed, swim speed).

If a DLL simply sets the CTM Action to 0x0D⁴, the client continues to generate MSG_MOVE_HEARTBEAT (0x00EE) packets because the physics engine is still moving the character forward. However, the DLL might simultaneously be trying to cast a spell or interact with an object. The server receives a heartbeat indicating movement, followed immediately by a spellcast request that requires the character to be stationary. The server immediately rejects the spellcast, flags the kinetic anomaly, and if the discrepancy persists, the server-side anti-cheat heuristics will forcefully disconnect the client for suspected teleportation or speed-hacking manipulation.

Strategic Implications for C++ DLL Injection

Executing internal client functions such as ClickToMove (0x00727400) and StopCTM (0x0072B3A0)⁵ from an injected dynamic-link library necessitates rigorous adherence to thread safety protocols and execution context awareness. The World of Warcraft 3.3.5a client is predominantly single-threaded concerning its core game logic, Object Manager traversal, and UI rendering layers.¹

Thread Safety and Execution Context

If an external application or an asynchronously injected thread attempts to call StopCTM arbitrarily, it introduces severe race conditions. For example, if the custom thread clears the MovementFlags at the exact millisecond the main game loop is iterating over the spline interpolation logic, it can trigger memory access violations (0xC0000005), immediately crashing the client.

To guarantee absolute deterministic behavior, all calls to CGPlayer_C movement functions must be executed synchronously from within the context of the client's main execution thread. In DirectX 9 environments, which serves as the graphical API foundation for build 12340, this is universally achieved by hooking the EndScene or Present functions of the IDirect3DDevice9 interface.

By hijacking the execution flow during the final rendering phase of a frame, the injected DLL ensures that the game engine is in a paused, stable state. The state machine is frozen, allowing the injected logic to safely evaluate IsClickMoving (0x00721F90)⁵, issue StopCTM (0x0072B3A0), and allow the client to safely reconstruct its network payloads before the subsequent frame is processed.

Handling Orientation and Facing

In many automated routing scenarios—such as those implemented in Fishbot-3.3.5¹¹ or combat rotation engines¹—merely stopping the character is insufficient; the character must also physically face a specific entity or node to satisfy interaction cone requirements.

The LocalPlayer class provides a dedicated SetFacing function at 0x0072EA50.⁵ When a CTM trajectory is abruptly interrupted via StopCTM, the character's orientation (yaw) remains statically frozen at the exact angle of the interruption. If subsequent logic dictates an interaction (such as utilizing the Interact function pointer at offset 44⁵), combining StopCTM with a synchronous call to SetFacing ensures the character model behaves naturally, adhering strictly to the expected visual and server-side interaction vectors.

UI Dependencies and Configuration Variables (CVars)

While this forensic analysis focuses heavily on physical memory and ASM-level engine interruptions, it is imperative to address the interaction between the CTM subsystem and the client's high-level user interface configuration variables (CVars). Developers often conflate UI-level toggles with engine-level state machines.

The Autointeract CVar

The client stores a boolean configuration variable known as autointeract, which governs whether the user interface is permitted to issue CTM commands based on mouse input.¹⁰ Players frequently execute Lua macros such as /run C_CVar.SetCVar("autointeract", 1) to toggle this accessibility functionality.¹⁰ Users often report bugs where this setting becomes unchecked upon zone transitions or UI reloads.¹⁷

However, from the perspective of an injected DLL directly calling 0x00727400⁵, the autointeract CVar is fundamentally irrelevant. The internal CTM function pointer executes the mathematical and state-machine operations directly, bypassing the UI-layer checks entirely. Consequently, toggling CVars or executing Lua commands like InterfaceOptionsMousePanelClickToMove:Click()¹⁰ will never interrupt an active movement spline generated by a direct memory call. The CVar solely dictates whether a hardware mouse click on the DirectX render window should translate into a ClickToMove invocation. It exercises zero authority over the physical spline engine.

Add-on Interferences and Frame Scripting

Similarly, user interface add-ons that attempt to manage tooltips or world object interactions¹⁹ frequently hook into the CTM UI logic. While these add-ons can pollute the global Lua namespace or cause visual glitches¹⁸, they cannot alter the execution of StopCTM (0x0072B3A0).⁵ The memory-level teardown sequence operates beneath the Lua scripting

engine, providing absolute and unmitigated control over the client's state regardless of the user's interface configuration.

Implications for Botting and Automation Frameworks

The ability to cleanly interrupt movement is the foundational requirement for any advanced automation framework operating within the 3.3.5a ecosystem. Projects such as IceFlake⁵, Python-based WotLKRotations¹, and Fishbot-3.3.5¹¹ rely heavily on dynamic memory reading to ascertain the state of the Object Manager.

In a sophisticated combat rotation engine, the DLL continuously reads dynamic object data—such as health, power, absolute position, status flags, and known spell IDs—directly from memory.¹ If the rule-based engine determines that the optimal action is to cast a channeled spell (e.g., verifying ChanneledCastingId at 0xA80 or CastingId at 0xA6C⁵), the character must be completely stationary. If the DLL simply alters the CTM action to 0x0D⁴ and attempts to invoke a spellcast, the server's tracking of the FORWARD flag¹² will instantly interrupt the cast. By implementing the StopCTM routine meticulously, these frameworks ensure that the local kinetic state precisely mirrors the requirements of the spellcasting engine, facilitating flawless programmatic execution.

Conclusion

The integration of autonomous programmatic movement within the World of Warcraft 3.3.5a client represents a highly complex interplay between mathematical spline interpolation, bitwise state flag management, and continuous asynchronous network synchronization. The persistent failure to halt the player character using high-level state mutations—such as overwriting the CTM action to 0x0D or attempting to overwrite the destination vector with an identical position—occurs because these methodologies address only the superficial intent of the state machine, completely ignoring the underlying physical trajectory calculations and network replication layers.

The indefinite transmission of MSG_MOVE_HEARTBEAT (0x00EE) packets serves as the primary diagnostic indicator of this partial state mutation. The client architecture recognizes that the overarching task is "Idle," yet the physics engine, governed strictly by the MovementInfo structure and the persistent FORWARD flag (0x00000001)¹², continues to assert active spatial displacement.

To achieve absolute programmatic parity with the client's intended design, software engineers must utilize the native StopCTM internal function located at memory address 0x0072B3A0.⁵ This function operates as a holistic and uncompromising teardown mechanism. It deallocates the pathing spline, resets the CTM memory block parameters (including the Action at 0x1C and the GUID at 0x20)⁴, forcefully zeroes the appropriate MovementFlags¹², and most critically, correctly packages the exact local coordinates and timestamps into a MSG_MOVE_STOP

(0x00CF)² network opcode.

By executing this function—ideally preceded by state validation via `IsClickMoving` at 0x00721F90⁵—from a thread-safe execution context such as an `EndScene` DirectX hook, developers can guarantee absolute synchronization between the local client prediction engine and the authoritative server architecture. While manual manipulation of the `MovementInfo` bitmasks remains a theoretical possibility, it is fraught with extreme architectural pitfalls regarding network payload construction and timestamp validation, unequivocally rendering the direct invocation of `StopCTM` the singular, definitive, and structurally sound solution for movement interruption in build 12340.

Works cited

1. AzDeltaQQ/WotLKRotations: A Python-based experimental framework for interacting with World of Warcraft (3.3.5a - 12340 client) memory to monitor game state and potentially execute combat rotations. (Extremely Work-in-progress) - GitHub, accessed February 20, 2026, <https://github.com/AzDeltaQQ/WotLKRotations>
2. `server/src/game/Server/Opcodes.h` at master - GitHub, accessed February 20, 2026, <https://github.com/mangoszero/server/blob/master/src/game/Server/Opcodes.h>
3. `WowPacketParser/WowPacketParser/Enums/Version/V4_0_3_13329/Opcodes.cs` at master · TrinityCore/WowPacketParser - GitHub, accessed February 20, 2026, https://github.com/TrinityCore/WowPacketParser/blob/master/WowPacketParser/Enums/Version/V4_0_3_13329/Opcodes.cs
4. Click To Move - WowDev wiki, accessed February 20, 2026, https://wowdev.wiki/Click_To_Move
5. `IceFlake/IceFlake/Client/Patchables/Pointers.cs` at master · miceiken ..., accessed February 20, 2026, <https://github.com/miceiken/IceFlake/blob/master/IceFlake/Client/Patchables/Pointers.cs>
6. Need help to stop clicking ! Some questions... : r/classicwow - Reddit, accessed February 20, 2026, https://www.reddit.com/r/classicwow/comments/1i3oj7o/need_help_to_stop_clicking_some_questions/
7. How to Stop Keyboard Turning and Clicking! An in Depth Guide for World of Warcraft and other MMO's - YouTube, accessed February 20, 2026, <https://www.youtube.com/watch?v=VE7NT8Hxy9k>
8. 3.3.5 camera bug : r/wowservers - Reddit, accessed February 20, 2026, https://www.reddit.com/r/wowservers/comments/1f5b9mg/335_camera_bug/
9. How Can I Unregister Clicks From MainMenuBar/Actionbars? - Blizzard Forums, accessed February 20, 2026, <https://us.forums.blizzard.com/en/wow/t/how-can-i-unregister-clicks-from-main-menubaractionbars/1764648>
10. Toggle on/off Click to Move - UI and Macro - World of Warcraft Forums, accessed

February 20, 2026,

<https://us.forums.blizzard.com/en/wow/t/toggle-onoff-click-to-move/959251>

11. WowDevs/Fishbot-3.3.5: World of Warcraft 3.3.5a Fishing Bot - GitHub, accessed February 20, 2026, <https://github.com/WowDevs/Fishbot-3.3.5>
12. SMSG UPDATE OBJECT - WowDev wiki, accessed February 20, 2026, https://wowdev.wiki/SMSG_UPDATE_OBJECT
13. EnumeratedString - wowdev, accessed February 20, 2026, <https://wowdev.wiki/EnumeratedString>
14. Mangos芒果魔兽世界法术坐骑光环BUFF DEBUFF 系统原创 - CSDN博客, accessed February 20, 2026, <https://blog.csdn.net/liuyinxing/article/details/134775764>
15. ElunaTrinityWotlk/src/server/game/Entities/Creature/Creature.cpp at master - GitHub, accessed February 20, 2026, <https://github.com/ElunaLuaEngine/ElunaTrinityWotlk/blob/master/src/server/game/Entities/Creature/Creature.cpp>
16. "Click-to-move" constantly becomes unchecked - Page 3 - Bug Report - World of Warcraft Forums, accessed February 20, 2026, <https://us.forums.blizzard.com/en/wow/t/click-to-move-constantly-becomes-unchecked/1501820?page=3>
17. "Click-to-move" constantly becomes unchecked - Bug Report - World of Warcraft Forums, accessed February 20, 2026, <https://us.forums.blizzard.com/en/wow/t/click-to-move-constantly-becomes-unchecked/1501820>
18. Click-to-move disables at every launch : r/wotlk - Reddit, accessed February 20, 2026, https://www.reddit.com/r/wotlk/comments/18fs6b0/clicktomove_disables_at_every_launch/
19. Tooltip of NPC / World objects stay active · Issue #53 · someweirdhuman/awesome_wotlk, accessed February 20, 2026, https://github.com/someweirdhuman/awesome_wotlk/issues/53